Подробный гайд

по созданию Telegram-бота для пиццерии на Python с использованием `aiogram 3.x` и `sqlite3`. Код разбит на модули, организованные поэтапно.

⚠️ **ВНИМАНИЕ**: Инструкция по регистрации бота через @BotFather и способом получения токена находится в самом конце данного гайда.

📂 Структура проекта

```
pizza_bot_aiogram/
├── bot.py                  # Точка входа
├── config.py               # Токен и константы
├── database.py             # Работа с SQLite
├── keyboards.py            # Клавиатуры
├── handlers/
│   ├── __init__.py
│   ├── start.py            # Регистрация и главное меню
│   ├── menu.py             # Выбор пиццы, размера, начинок
│   ├── order.py            # Оформление, адрес, комментарий
│   ├── cancel.py           # Отмена заказа
│   └── feedback.py         # Отзывы
├── states.py               # FSM состояния
└── utils.py                # Вспомогательные функции
```

1. Установка зависимостей

```
pip install aiogram
```

2. Файл `config.py` – настройки

```python
TOKEN = "ВАШ_ТОКЕН_БОТА"  # Получить у @BotFather
OWNER_ID = 123456789      # Telegram ID владельца пиццерии

# Время доставки в минутах
DELIVERY_TIME = 45

# База данных
DB_NAME = "pizza_bot.db"
```

3. Файл `database.py` – работа с SQLite

```python
import sqlite3
import datetime
from config import DB_NAME

def init_db():
    """Инициализация базы данных"""
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()

    # Таблица пользователей
    cur.execute('''
        CREATE TABLE IF NOT EXISTS users (
            user_id INTEGER PRIMARY KEY,
            username TEXT,
            first_name TEXT,
            phone TEXT,
            address TEXT,
            registered_at TEXT
        )
    ''')

    # Таблица заказов
    cur.execute('''
        CREATE TABLE IF NOT EXISTS orders (
            order_id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER,
            items TEXT,
            total_price REAL,
            address TEXT,
            comment TEXT,
            status TEXT,
            created_at TEXT,
            FOREIGN KEY (user_id) REFERENCES users (user_id)
        )
    ''')

    # Таблица отзывов
    cur.execute('''
        CREATE TABLE IF NOT EXISTS feedback (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER,
            rating INTEGER,
            comment TEXT,
            created_at TEXT
        )
    ''')

    conn.commit()
    conn.close()

def add_user(user_id, username, first_name):
    """Добавление нового пользователя"""
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()
    cur.execute('''
        INSERT OR IGNORE INTO users (user_id, username, first_name, registered_at)
```

```
        VALUES (?, ?, ?, ?)
    ''', (user_id, username, first_name, datetime.datetime.now().isoformat()))
    conn.commit()
    conn.close()

def update_user_address(user_id, address):
    """Обновление адреса пользователя"""
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()
    cur.execute("UPDATE users SET address = ? WHERE user_id = ?", (address, user_id))
    conn.commit()
    conn.close()

def save_order(user_id, items, total_price, address, comment):
    """Сохранение заказа в БД"""
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()
    cur.execute('''
        INSERT INTO orders (user_id, items, total_price, address, comment, status, created_at)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', (user_id, items, total_price, address, comment, "Новый", datetime.datetime.now().isoformat()))
    order_id = cur.lastrowid
    conn.commit()
    conn.close()
    return order_id

def get_pending_order(user_id):
    """Получение активного (не отменённого) заказа пользователя"""
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()
    cur.execute('''
        SELECT order_id, items, total_price, address, comment, status
        FROM orders
        WHERE user_id = ? AND status = 'pending'
        ORDER BY order_id DESC LIMIT 1
    ''', (user_id,))
    row = cur.fetchone()
    conn.close()
    return row

def cancel_order(order_id):
    """Отмена заказа"""
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()
    cur.execute("UPDATE orders SET status = 'cancelled' WHERE order_id = ?", (order_id,))
    conn.commit()
    conn.close()

def save_feedback(user_id, rating, comment):
    """Сохранение отзыва"""
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()
    cur.execute('''
        INSERT INTO feedback (user_id, rating, comment, created_at)
        VALUES (?, ?, ?, ?)
    ''', (user_id, rating, comment, datetime.datetime.now().isoformat()))
    conn.commit()
    conn.close()
```

4. Файл `states.py` – состояния FSM

```python
from aiogram.fsm.state import State, StatesGroup

class OrderStates(StatesGroup):
    """Состояния для оформления заказа"""
    waiting_for_address = State()
    waiting_for_comment = State()

class FeedbackStates(StatesGroup):
    """Состояния для отзыва"""
    waiting_for_rating = State()
    waiting_for_comment = State()
```

5. Файл `menu_data.py` – данные меню

```python
# Базовая цена в зависимости от размера
BASE_PRICES = {
    "Маргарита": {"маленький": 250, "средний": 350, "большой": 450},
    "Маринара": {"маленький": 230, "средний": 330, "большой": 430}
}

# Дополнительные начинки
TOPPINGS = {
    "колбаса": 50,
    "салями": 60,
    "сыр": 40
}

def calculate_price(pizza_name: str, size: str, toppings: list) -> int:
    """Расчёт стоимости заказа"""
    price = BASE_PRICES[pizza_name][size]
    for topping in toppings:
        price += TOPPINGS.get(topping, 0)
    return price
```

6. Файл `keyboards.py` – клавиатуры

```python
from aiogram.types import ReplyKeyboardMarkup, KeyboardButton, InlineKeyboardMarkup, InlineKeyboardButton

def main_keyboard():
    """Главная клавиатура"""
    keyboard = ReplyKeyboardMarkup(
        keyboard=[
            [KeyboardButton(text="🍕 Меню")],
            [KeyboardButton(text="📝 Оставить отзыв"), KeyboardButton(text="❌ Отменить заказ")]
        ],
```

```python
        resize_keyboard=True
    )
    return keyboard

def pizza_menu_keyboard():
    """Клавиатура выбора пиццы"""
    keyboard = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="🍕 Маргарита", callback_data="pizza_Маргарита")],
        [InlineKeyboardButton(text="🍕 Маринара", callback_data="pizza_Маринара")]
    ])
    return keyboard

def size_keyboard(pizza_name: str):
    """Клавиатура выбора размера"""
    keyboard = InlineKeyboardMarkup(inline_keyboard=[
        [
            InlineKeyboardButton(text="🟢 Маленький", callback_data=f"size_{pizza_name}_маленький"),
            InlineKeyboardButton(text="🟡 Средний", callback_data=f"size_{pizza_name}_средний"),
            InlineKeyboardButton(text="🔴 Большой", callback_data=f"size_{pizza_name}_большой")
        ]
    ])
    return keyboard

def toppings_keyboard(current_toppings: list):
    """Клавиатура выбора начинок"""
    buttons = []
    for topping, price in [("колбаса", 50), ("салями", 60), ("сыр", 40)]:
        check = "✅ " if topping in current_toppings else ""
        buttons.append([InlineKeyboardButton(
            text=f"{check}{topping} (+{price}₽)",
            callback_data=f"topping_{topping}"
        )])

    buttons.append([InlineKeyboardButton(text="✅ Готово", callback_data="toppings_done")])
    buttons.append([InlineKeyboardButton(text="❌ Отменить заказ", callback_data="cancel_order")])

    return InlineKeyboardMarkup(inline_keyboard=buttons)

def rating_keyboard():
    """Клавиатура для оценки"""
    buttons = [[InlineKeyboardButton(text=str(i), callback_data=f"rating_{i}") for i in range(1, 6)]]
    return InlineKeyboardMarkup(inline_keyboard=buttons)
```

7. Файл `handlers/start.py` – регистрация и главное меню

```python
from aiogram import Router, F
from aiogram.types import Message
from aiogram.filters import CommandStart
from database import add_user
from handlers.keyboards import main_keyborad

router = Router()

@router.message(CommandStart())
async def cmd_start(message: Message):
```

```
    """Обработчик команды /start - регистрация пользователя"""
    user_id = message.from_user.id
    username = message.from_user.username
    username = "без имени" if username is None else username
    first_name = message.from_user.first_name
    first_name = 'Гость' if first_name is None else first_name

    # Регистрация в БД
    add_user(user_id, username, first_name)

    await message.answer(
        f"🍕 Добро пожаловать в PizzaBot, {first_name}!\n\n"
        f"Здесь вы можете заказать вкусную пиццу с доставкой.\n"
        f"Выберите действие:",
        reply_markup=main_keyboard()
    )
```

8. Файл `handlers/menu.py` – выбор блюда

```
from aiogram import Router, F
from aiogram.types import Message, CallbackQuery
from aiogram.fsm.context import FSMContext
from handlers.keyboards import pizza_menu_keyboard, size_keyboard, toppings_keyboard
from menu_data import BASE_PRICES, TOPPINGS, calculate_price

router = Router()

# Хранилище временных данных пользователя
user_orders = {}

@router.message(F.text == "🍕 Меню")
async def show_menu(message: Message):
    """Показать меню пицц"""
    await message.answer("🍕 Выберите пиццу:", reply_markup=pizza_menu_keyboard())

@router.callback_query(F.data.startswith("pizza_"))
async def choose_pizza(callback: CallbackQuery):
    """Выбор пиццы"""
    pizza_name = callback.data.split("_")[1]
    user_id = callback.from_user.id

    # Инициализируем данные заказа
    user_orders[user_id] = {
        "pizza": pizza_name,
        "size": None,
        "toppings": []
    }

    await callback.message.edit_text(
        f"Вы выбрали: {pizza_name}\n\nВыберите размер:",
        reply_markup=size_keyboard(pizza_name)
    )
    await callback.answer()

@router.callback_query(F.data.startswith("size_"))
```

```python
async def choose_size(callback: CallbackQuery):
    """Выбор размера"""
    _, pizza_name, size = callback.data.split("_")
    user_id = callback.from_user.id

    if user_id not in user_orders:
        user_orders[user_id] = {}

    user_orders[user_id]["pizza"] = pizza_name
    user_orders[user_id]["size"] = size

    current_toppings = user_orders[user_id].get("toppings", [])

    await callback.message.edit_text(
        f"📋 Ваш заказ:\n"
        f"Пицца: {pizza_name}\n"
        f"Размер: {size}\n\n"
        f"➕ Добавьте начинки (можно несколько):",
        reply_markup=toppings_keyboard(current_toppings)
    )
    await callback.answer()

@router.callback_query(F.data.startswith("topping_"))
async def add_topping(callback: CallbackQuery):
    """Добавление начинки"""
    topping = callback.data.split("_")[1]
    user_id = callback.from_user.id

    if topping == "done":
        # Переходим к оформлению заказа
        await callback.message.delete()
        await proceed_to_order(callback.message, user_id)
        return

    # Добавляем или удаляем начинку
    if topping in user_orders[user_id]["toppings"]:
        user_orders[user_id]["toppings"].remove(topping)
        await callback.answer(f"❌ {topping} удалена")
    else:
        user_orders[user_id]["toppings"].append(topping)
        await callback.answer(f"✅ {topping} добавлена")

    # Обновляем клавиатуру
    current_toppings = user_orders[user_id]["toppings"]
    await callback.message.edit_reply_markup(reply_markup=toppings_keyboard(current_toppings))

@router.callback_query(F.data == "toppings_done")
async def finish_toppings(callback: CallbackQuery):
    """Завершение выбора начинок"""
    user_id = callback.from_user.id
    await callback.message.delete()
    await proceed_to_order(callback.message, user_id)
    await callback.answer()

async def proceed_to_order(message: Message, user_id: int):
    """Переход к оформлению заказа"""
    from handlers.order import start_order_process
    await start_order_process(message, user_id, user_orders[user_id])
```

9. Файл `handlers/order.py` – оформление заказа

```python
from aiogram import Router, F
from aiogram.types import Message, CallbackQuery
from aiogram.fsm.context import FSMContext
from config import OWNER_ID, DELIVERY_TIME
from database import save_order, update_user_address
from states import OrderStates
from menu_data import calculate_price

router = Router()

async def start_order_process(message: Message, user_id: int, order_data: dict, state: FSMContext):
    """Начать процесс оформления заказа"""
    # Рассчитываем стоимость
    total_price = calculate_price(
        order_data["pizza"],
        order_data["size"],
        order_data["toppings"]
    )

    # Сохраняем данные в состояние
    await state.update_data(
        pizza=order_data["pizza"],
        size=order_data["size"],
        toppings=order_data["toppings"],
        total_price=total_price
    )

    # Формируем описание заказа
    toppings_text = ", ".join(order_data["toppings"]) if order_data["toppings"] else "без начинок"

    await message.answer(
        f"🛒 Ваш заказ:\n"
        f"🍕 {order_data['pizza']} ({order_data['size']})\n"
        f"➕ Начинки: {toppings_text}\n"
        f"💰 Сумма: {total_price}₽\n\n"
        f"📍 Пожалуйста, введите адрес доставки:"
    )

    await state.set_state(OrderStates.waiting_for_address)

@router.message(OrderStates.waiting_for_address)
async def get_address(message: Message, state: FSMContext):
    """Получение адреса доставки"""
    address = message.text
    user_id = message.from_user.id

    # Сохраняем адрес в БД
    update_user_address(user_id, address)

    # Сохраняем адрес в состояние
    await state.update_data(address=address)

    await message.answer(
        "📝 Добавьте комментарий к заказу\n"
        "(например: 'Без соли', 'Острая', 'Побольше сыра')\n"
        "Или напишите 'нет', если комментарий не нужен:"
```

```
    )

    await state.set_state(OrderStates.waiting_for_comment)

@router.message(OrderStates.waiting_for_comment)
async def get_comment(message: Message, state: FSMContext):
    """Получение комментария и сохранение заказа"""
    comment = message.text if message.text.lower() != "нет" else ""
    user_id = message.from_user.id

    # Получаем все данные из состояния
    data = await state.get_data()

    # Формируем строку с заказом
    toppings_text = ", ".join(data["toppings"]) if data["toppings"] else "без начинок"
    items_text = f"{data['pizza']} ({data['size']}) + {toppings_text}"

    # Сохраняем заказ в БД
    order_id = save_order(
        user_id,
        items_text,
        data["total_price"],
        data["address"],
        comment
    )

    # Отправляем подтверждение пользователю
    await message.answer(
        f"✅ Заказ №{order_id} принят!\n\n"
        f"⏱️ Ожидаемое время доставки: {DELIVERY_TIME} минут\n"
        f"💰 Итоговая сумма: {data['total_price']}₽\n\n"
        f"Спасибо за заказ! ❤️"
    )

    # Отправляем уведомление владельцу
    owner_text = (
        f"🔔 НОВЫЙ ЗАКАЗ №{order_id}\n\n"
        f"👤 Клиент: @{message.from_user.username or message.from_user.first_name}\n"
        f"📦 Заказ: {items_text}\n"
        f"💰 Сумма: {data['total_price']}₽\n"
        f"📍 Адрес: {data['address']}\n"
        f"📝 Комментарий: {comment if comment else 'Нет'}"
    )

    from config import OWNER_ID
    from bot import bot
    await bot.send_message(OWNER_ID, owner_text)

    # Очищаем состояние
    await state.clear()
```

10. Файл `handlers/cancel.py` – отмена заказа

```
from aiogram import Router, F
from aiogram.types import Message
```

```python
from database import get_pending_order, cancel_order
from config import OWNER_ID

router = Router()

@router.message(F.text == "❌ Отменить заказ")
async def cancel_order_handler(message: Message):
    """Отмена активного заказа"""
    user_id = message.from_user.id

    # Проверяем наличие активного заказа
    order = get_pending_order(user_id)

    if not order:
        await message.answer("❌ У вас нет активных заказов для отмены.")
        return

    order_id = order[0]
    cancel_order(order_id)

    await message.answer(f"✅ Заказ №{order_id} успешно отменён.\n\nВы можете сделать новый заказ через меню.")

    # Уведомляем владельца
    from bot import bot
    await bot.send_message(
        OWNER_ID,
        f"⚠️ Заказ №{order_id} отменён пользователем @{message.from_user.username or message.from_user.first_na
    )
```

11. Файл `handlers/feedback.py` – отзывы

```python
from aiogram import Router, F
from aiogram.types import Message, CallbackQuery
from aiogram.fsm.context import FSMContext
from database import save_feedback
from states import FeedbackStates
from keyboards import rating_keyboard
from config import OWNER_ID

router = Router()

@router.message(F.text == "📝 Оставить отзыв")
async def ask_rating(message: Message, state: FSMContext):
    """Запрос оценки"""
    await message.answer(
        "⭐ Пожалуйста, оцените нашу работу от 1 до 5:",
        reply_markup=rating_keyboard()
    )
    await state.set_state(FeedbackStates.waiting_for_rating)

@router.callback_query(FeedbackStates.waiting_for_rating, F.data.startswith("rating_"))
async def get_rating(callback: CallbackQuery, state: FSMContext):
    """Получение оценки"""
    rating = int(callback.data.split("_")[1])
    await state.update_data(rating=rating)
```

```
    await callback.message.delete()
    await callback.message.answer(
        f"Спасибо за оценку {rating}⭐\n\n"
        f"Напишите ваш отзыв подробнее (или отправьте '-' чтобы пропустить):"
    )
    await state.set_state(FeedbackStates.waiting_for_comment)
    await callback.answer()

@router.message(FeedbackStates.waiting_for_comment)
async def get_feedback_comment(message: Message, state: FSMContext):
    """Получение текста отзыва"""
    user_id = message.from_user.id
    data = await state.get_data()
    rating = data.get("rating")

    comment = message.text if message.text != "-" else ""

    # Сохраняем отзыв
    save_feedback(user_id, rating, comment)

    await message.answer("🙏 Большое спасибо за ваш отзыв! Это помогает нам стать лучше.")

    # Уведомляем владельца
    from bot import bot
    owner_text = (
        f"⭐ НОВЫЙ ОТЗЫВ\n\n"
        f"👤 От: @{message.from_user.username or message.from_user.first_name}\n"
        f"⭐ Оценка: {rating}/5\n"
        f"📝 Текст: {comment if comment else 'Без комментария'}"
    )
    await bot.send_message(OWNER_ID, owner_text)

    await state.clear()
```

12. Файл `bot.py` – точка входа

```
import asyncio
import logging
from aiogram import Bot, Dispatcher
from config import TOKEN
from database import init_db
from handlers import start, menu, order, cancel, feedback

# Настройка логирования
logging.basicConfig(level=logging.INFO)

async def main():
    """Главная функция запуска бота"""
    # Инициализация базы данных
    init_db()

    # Создание бота и диспетчера
    bot = Bot(token=TOKEN)
    dp = Dispatcher()
```

```
    # Подключение роутеров
    dp.include_router(start.router)
    dp.include_router(menu.router)
    dp.include_router(order.router)
    dp.include_router(cancel.router)
    dp.include_router(feedback.router)

    # Запуск бота
    await bot.delete_webhook(drop_pending_updates=True)
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())
```

13. Файл `handlers/__init__.py`

```
# Пустой файл для обозначения пакета
```

🚀 Запуск бота

```
python bot.py
```

🎯 Особенности данного решения

- ✅ **FSM (Finite State Machine)** – правильная обработка диалогов
- ✅ **Модульная структура** – легко расширять и поддерживать
- ✅ **Inline-клавиатуры** – удобный выбор опций
- ✅ **Хранение состояния** – данные пользователя не теряются
- ✅ **Двойные уведомления** – клиент + владелец
- ✅ **Отмена заказа** – проверка активных заказов

🔧 Возможные доработки для продвинутой части

1. **Панель администратора** – добавление и редактирование заказов, просмотр всех заказов, изменение статуса
2. **Платежи** – интеграция с ЮKassa или Stripe
3. **Учет скидок и акций** - например, скидка 10% на второй заказ
4. **Статусы заказа** – "готовится", "в пути", "доставлен"

5. **История заказов** – кнопка "Мои заказы"
6. **Повтор заказа** – кнопка "Заказать снова"
7. **Корзина** – несколько позиций в одном заказе

📌 Важные замечания

1. Замените `TOKEN` и `OWNER_ID` в `config.py` на свои значения
2. Для получения `OWNER_ID` напишите боту `@userinfobot`
3. При первом запуске база данных создастся автоматически
4. Бот использует `aiogram 3.x` – последнюю версию фреймворка

📘 Мини-гайд: Регистрация бота в Telegram и получение токена + ID владельца

🤖 Часть 1. Создание бота и получение токена через @BotFather

Шаг 1. Найти BotFather

1. Откройте Telegram
2. В строке поиска введите `@BotFather`
3. Нажмите на официального бота с галочкой ✅

Шаг 2. Создать нового бота

Отправьте BotFather команду:

```
/newbot
```

Шаг 3. Выбрать имя бота

Введите **отображаемое имя** вашего бота (может быть на русском):

```
Пиццерия Бот
```

> Это имя пользователи будут видеть в чате

Шаг 4. Выбрать username бота

Введите **уникальный username** (только латиница, обязательно заканчивается на `bot`):

```
pizzeria_delivery_bot
```

> Примеры: `my_pizza_bot`, `pizza_house_bot`

Шаг 5. Получить токен

После успешного создания BotFather ответит сообщением с **токеном**:

```
Done! Congratulations on your new bot. You will find it at t.me/pizzeria_delivery_bot

Use this token to access the HTTP API:
1234567890:AAFv_abcdefghijklmnopqrstuvwxyz-12345
```

🎉 **Это ваш токен!** Сохраните его в безопасном месте.

🆔 Часть 2. Получение ID владельца (OWNER_ID)

Вам понадобится ваш личный Telegram ID, чтобы бот присылал уведомления о заказах.

Способ 1. Через бота @userinfobot (САМЫЙ ПРОСТОЙ)

1. Найдите в Telegram бота: `@userinfobot`
2. Нажмите **Start** или отправьте команду `/start`
3. Бот ответит сообщением с вашими данными:

```
🆔 Your user ID: 123456789
👤 First name: Иван
🇷🇺 Last name: Петров
💬 Username: @ivan_petrov
```

👉 **Цифра после "Your user ID" – это ваш OWNER_ID**

Способ 2. Через любого своего бота (временный код)

Если у вас уже есть бот, отправьте ему команду `/start` и добавьте этот код:

```
@bot.message_handler(commands=['start'])
def get_id(message):
    bot.send_message(message.chat.id, f"Ваш ID: {message.from_user.id}")
```

Способ 3. Через @get_id_bot

1. Найдите бота `@get_id_bot`
2. Нажмите Start
3. Бот покажет ваш ID

📝 Часть 3. Что делать с токеном и ID

Вставьте их в файл `config.py` :

```
# config.py
TOKEN = "1234567890:AAFv_abcdefghijklmnopqrstuvwxyz-12345"  # Ваш токен от BotFather
OWNER_ID = 123456789  # Ваш ID от @userinfobot
```

Пример готового файла:

```
TOKEN = "7203456789:AAGmE_xyzXYZabcABC1234567890-abcde"
OWNER_ID = 987654321
DELIVERY_TIME = 45
DB_NAME = "pizza_bot.db"
```

⚠️ Важные правила безопасности

❌ НЕ ДЕЛАЙТЕ	✅ ДЕЛАЙТЕ
Не публикуйте токен в интернете	Храните токен в `config.py`
Не загружайте токен на GitHub	Добавьте `config.py` в `.gitignore`
Не отправляйте токен в чаты	Используйте переменные окружения
Не показывайте токен на скриншотах	Делайте ревью кода перед публикацией

🔧 Альтернатива: переменные окружения (для продвинутых)

Вместо хранения токена в файле, используйте переменные окружения:

Windows (CMD):

```
set BOT_TOKEN=1234567890:AAFv_abcdefghijklmnopqrstuvwxyz-12345
set OWNER_ID=987654321
```

Windows (PowerShell):

```
$env:BOT_TOKEN="1234567890:AAFv_abcdefghijklmnopqrstuvwxyz-12345"
$env:OWNER_ID="987654321"
```

Linux/Mac:

```
export BOT_TOKEN="1234567890:AAFv_abcdefghijklmnopqrstuvwxyz-12345"
export OWNER_ID="987654321"
```

В коде:

```
import os
TOKEN = os.getenv("BOT_TOKEN")
OWNER_ID = int(os.getenv("OWNER_ID"))
```

🧪 Часть 4. Проверка работоспособности

Быстрый тестовый скрипт:

```
from config import TOKEN, OWNER_ID
import requests

# Проверка токена
url = f"https://api.telegram.org/bot{TOKEN}/getMe"
response = requests.get(url)

if response.status_code == 200:
    data = response.json()
    if data["ok"]:
        print(f"✅ Бот работает: @{data['result']['username']}")
    else:
        print("❌ Неверный токен")
else:
    print("❌ Ошибка подключения")

print(f"👑 ID владельца: {OWNER_ID}")
```

Ожидаемый вывод:

```
✅ Бот работает: @pizzeria_delivery_bot
👑 ID владельца: 987654321
```

📸 Скриншоты для наглядности

BotFather после создания бота:

```
🤖 BotFather, [дата]

Done! Congratulations on your new bot.

Use this token to access the HTTP API:
7203456789:AAGmE_xyzXYZabcABC1234567890-abcde

Bot username: @pizzeria_delivery_bot
```

@userinfobot ответ:

```
🆔 Your user ID: 987654321
👤 First name: Иван
💬 Username: @ivan
```

🆘 Часто задаваемые вопросы

Вопрос: Бот не запускается, пишет "Unauthorized"
Ответ: Вы скопировали токен с ошибкой. Убедитесь, что в нём ровно столько символов, сколько выдал BotFather.

Вопрос: Как сменить имя бота?
Ответ: Напишите BotFather команду `/mybots`, выберите бота → Edit Bot → Edit Name

Вопрос: Как получить ID другого пользователя?
Ответ: Попросите его написать @userinfobot или переслать любое сообщение от него вашему боту

Вопрос: Токен скомпрометирован. Что делать?
Ответ: Напишите BotFather → `/mybots` → выберите бота → API Token → Revoke token

✅ Итоговый чек-лист

- [ ] Создан бот через @BotFather

- [ ] Получен и сохранён токен
- [ ] Найден свой ID через @userinfobot
- [ ] Токен и ID добавлены в `config.py`
- [ ] Токен не загружен на GitHub
- [ ] Тестовый скрипт показал ✅

Готово! Теперь вы можете использовать эти данные в боте для пиццерии 🍕